COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/DATA STRUCTURES AND ALGORITHM	**SECTION:**	CPE21S1
DATE PERFORMED:	28/7/2026	**DATE SUBMITTED:**	1/8/2026
NAME:	LOREEN JAE D. PILAPIL	**INSTRUCTOR:**	ENGR. QUEJADO

ACTIVITY NO. 4.1 STACKS

1. PROCEDURE OUTPUT

ILO A: STACKS USING STL

CODE:

main.cpp > main()

```cpp
#include<iostream>
#include<stack>

int main(){
    std::stack<int> newStack;

    std::cout<<"stack empty?"<<newStack.empty()<<std::endl;

    newStack.push(3);
    newStack.push(8);
    newStack.push(15);

    std::cout<<"stack empty? "<<newStack.empty()<<std::endl;
    std::cout<<"the size of the stack: "<<newStack.size()<<std::endl;
    std::cout<<"the top of the stack: "<<newStack.top()<<std::endl;
    std::cout<<"Removing elements from the stack: "<<std::endl;
    newStack.pop();
    std::cout<<"the top of the stack: "<<newStack.top()<<std::endl;
    newStack.pop();
    std::cout<<"the top of the stack: "<<newStack.top()<<std::endl;
    newStack.pop();
    std::cout<<"stack empty? "<<newStack.empty()<<std::endl;
    return 0;
}
```

OUTPUT

```
soft-MIEngine-Pid-1rk5ogdw.5cs' '--dbgExe=C:\msys64\ucrt64\bin\
stack empty?1
stack empty? 0
the size of the stack: 3
the top of the stack: 15
Removing elements from the stack:
the top of the stack: 8
the top of the stack: 3
stack empty? 1
```

ILO B: B.1 STACK USING ARRAY

CODE

main.cpp > Display()

```cpp
#include<iostream>

const size_t maxCap = 100;
int stack[maxCap];
int top = -1, i, newData;


void push();
void Pop();
void Top();
bool isEmpty();
bool isFull();
void Display();

int main(){
    int choice;
    std::cout<<"Give number of max elements for new stack: ";
    std::cin>>i;

    while(true){
        std::cout<<"=== Stack Operations ==="<<std::endl;
        std::cout<<"1. PUSH 2. POP 3. TOP 4. isEmpty 5.DISPLAY ALL"<<std::endl;
        std::cin>> choice;

        switch (choice){

            case 1: push();
            break;
            case 2: Pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout<< isEmpty()<<std::endl;
            break;
            case 5: Display();
            break;
            default: std::cout<<"invalid choice"<<std::endl;
            break;
        }
    }

    return 0;
}

bool isEmpty(){
    if(top == -1) return true;
    return false;
}
```

```
49
50  bool isFull(){
51      if(top > i-1) return true;
52      return false;
53  }
54
55  void push(){
56      if (top == i-1){
57          std::cout<<"stack overflow";
58          return;
59      }
60      std::cout<<"new value to add into the stack: "<<std::endl;
61      std::cin>> newData;
62
63      stack[++top]= newData;
64  }
65
66  void Top(){
67      if(isEmpty()){
68          std::cout<<"stack is empty"<<std::endl;
69      }
70      std::cout<<"the element on the top of the stack is "<< stack[top]<<std::endl;
71
72  }
73
74  void Pop(){
75      if(isEmpty()){
76          std::cout<<"stack undeflow"<<std::endl;
77      }
78      std::cout<<"Popping: "<< stack[top];
79      top--;
80  }
81
82  void Display(){
83      if(isEmpty()){
84          std::cout<<"Stack is empty"<<std::endl;
85      }
86      std::cout<<"Stack Elements: "<<std::endl;
87      for(int j = top; j>=0; j--){ //STARTS FROM THE TOP TO THE BOTTOM OF THE STACK
88          std::cout<<stack[j]<<""; // PRINTS STACK FROM TOP TO BOTTOM
89      }
90      std::cout << std::endl;
91  }
92
```

OUTPUT

```
soft-MIEngine-Pid-mtrpihwp.ow5' '--dbgExe=C:\msys64\ucrt64
Give number of max elements for new stack: 3
=== Stack Operations ===
1. PUSH 2. POP 3. TOP 4. isEmpty 5.DISPLAY ALL
1
new value to add into the stack:
3
=== Stack Operations ===
1. PUSH 2. POP 3. TOP 4. isEmpty 5.DISPLAY ALL
1
new value to add into the stack:
2
=== Stack Operations ===
1. PUSH 2. POP 3. TOP 4. isEmpty 5.DISPLAY ALL
1
new value to add into the stack:
1
=== Stack Operations ===
1. PUSH 2. POP 3. TOP 4. isEmpty 5.DISPLAY ALL
5
Stack Elements:
123
```

ANALYSIS

I modified the code to display all elements in the stack by first declaring void display() above. Then I added a switch case 5 to add an option on the menu. Then I added a function to check if the stack is empty. Then lastly I added a void Display() code below that loops the stack from top to bottom then proceeds to print it.

ILO B: B.2 STACKS USING LINKED LIST

CODE

C stacklist.h > pop<T>()

```
#ifndef STACK_LIST_H
#define STACK_LIST_H


template <typename T>
class Node{
    public:
    T data;
    Node* next;
};

template <typename T>
Node<T> *head = nullptr;

//push
template <typename T>
void push(T newData){
    Node<T>* newNode= new Node<T>;
    newNode->data = newData;
    newNode->next = head<T>;
    head<T> = newNode;

}
//pop
template <typename T>
T pop(){
    if(head<T> ==nullptr){ //check if stack is empty
        std::cout<<"Stack is empty"<<std::endl;
        return T();
    }
    Node<T>* temp = head<T>; //save current top node

    T tempVal= temp->data; //store data

    head<T> =head<T>->next;//move head

    delete temp;//free removed node

    return tempVal; //return removed value
}

//isEmpty
template <typename T>
bool isEmpty(){
    return head<T> ==nullptr;
}
```

```
47
48  //top
49  template <typename T>
50  void top(){
51      if(head<T> == nullptr){
52          std::cout<<"Stack undeflow"<<std::endl;
53          return;
54      }
55      std::cout<<"\n Top of stack: "<<head<T>->data<<std::endl;
56  }
57
58  //display all
59  template <typename T>
60  void display(){
61      if (head<T> == nullptr){
62          std::cout<<"stack underflow";
63      }
64      Node<T>* temp = head<T>;
65      while(temp != nullptr){
66          std::cout<<temp->data<<",";
67          temp = temp->next;
68      }
69      std::cout<<"\n";
70
71  }
72  #endif
```

main.cpp stacklist.h

main.cpp > main()

```cpp
#include <iostream>
#include "stacklist.h"

int main(){

    std::cout<< "--- PUSHING 'c' 'p' 'e' ---";

    push('c');
    push('p');
    push('e');

    top<char>();

    std::cout<<"\n --- POP all elements --- \n";
    while(!isEmpty<char>()){
        std::cout<<"Popped: "<<pop<char>()<<std::endl;

    }
    std::cout<<"\n --- Testing the Underflow --- \n";
    pop<char>();

    std::cout<<"\n --- Display all --- \n";
    display<char>();

    return 0;
}
```

OUTPUT

```
interpreter=mi'
--- PUSHING 'c' 'p' 'e' ---
 Top of stack: e

 --- POP all elements ---
Popped: e
Popped: p
Popped: c

 --- Testing the Underflow ---
Stack is empty

 --- Display all ---
stack underflow
PS C:\Users\TIPQC\OneDrive\Documents\CPE21S1\HOA4p1\ilob2>
```

ANALYSIS

On this part of the Hands on activity, we made a stack linked list then modified it to display all. First, we made a 'stacklist.h' file then we integrated push, pop, top, isEmpty, and display all to the code. Then on the main.cpp we initiated a code to display the output.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using an implementation of stack:

The following problem definition and algorithm is provided for checking the balancing of symbols. Create an implementation using stacks. Your output must include the following:

a. Stack using Arrays
b. Stack using Linked Lists
c. (Optional) Stack using C++ STL

Problem Definition:

Stacks can be used to check whether the given expression has balanced symbols. This algorithm is very useful in compilers. Each time the parser reads one character at a time. If the character is an opening delimiter such as (, {, or [- then it is written to the stack. When a closing delimiter is encountered like), }, or]-the stack is popped. The opening and closing delimiters are then compared. If they match, the parsing of the string continues. If they do not match, the parser indicates that there is an error on the line.

Steps:

1. Create a Stack.
2. While(end of input is not reached) {

a) If the character read is not a symbol to be balanced, ignore it.
b) If the character is an opening symbol, push it onto the stack.
c) If it is a closing symbol:
i) Report an error if the stack is empty.
ii) Otherwise, pop the stack.
d) If the symbol popped is not the corresponding opening symbol, report an error.

3. At the end of input, if the stack is not empty: report an error.
Self-Checking:
For the following cases, complete the table using the code you created

Expression	Valid? (Y/N)	Output	Analysis
(A+B)+(C-D)	Y	Enter an expression: (A+B)+(C-D) expression is valid.	Valid because every opening parenthesis has a matching closing parenthesis
((A+B)+(C-D)	N	Enter an expression: ((A+B)+(C-D) Error: Stack is not empty. expression is invalid	Invalid because there are two opening parentheses but only one closing parenthesis.
(A+B)+[C-D])	N	Enter an expression: (A+B)+[C-D]) Error: Stack is empty. expression is invalid	Invalid because the first '(' is matched correctly, and '[' is matched with ']'. But, there is one extra closing ')' with no corresponding opening '('.
(A+B]+[C-D]}	N	Enter an expression: (A+B]+[C-D]} Error: Symbols do not match. expression is invalid	Invalid because the first opening symbol is '(', but the first closing symbol is ']'.'[' is paired correctly with ']' but the last '}' does not match..

CODE

```cpp
// ILOC

    //read input expression
    std::cout << "Enter an expression: ";
    std::getline(std::cin, expression);

    // check whether the symbols are balanced
    if (iloc(expression)) {
        //opening and closing symbols matched
        std::cout << "expression is valid." << std::endl;
    } else {
        // incorrect symbol was found
        std::cout << "expression is invalid" << std::endl;
    }

    return 0;
}
```

```cpp
// ILOC
// Checks whether the symbols are balanced
bool iloc(std::string expression){

    // clear the stack
    while(!isEmpty<char>())
        pop<char>();

    // read each character
    for(char ch : expression){

        // ignore characters
        if(ch != '(' && ch != ')' &&
           ch != '{' && ch != '}' &&
           ch != '[' && ch != ']'){
            continue;
        }

        if(ch == '(' || ch == '{' || ch == '['){
            push<char>(ch); //push
        }

        // character is a closing symbol
        else{

            // If the stack is empty
            if(isEmpty<char>()){
                std::cout << "Error: Stack is empty." << std::endl;
                return false;
            }

            // pop the top opening symbol
            char topSymbol = pop<char>();

            // check if the symbols matches
            if((ch == ')' && topSymbol != '(') ||
               (ch == '}' && topSymbol != '{') ||
               (ch == ']' && topSymbol != '[')){
                std::cout << "Error: Symbols do not match." << std::endl;
                return false;
            }
        }
    }

    // if the stack is not empty
    if(!isEmpty<char>()){
        std::cout << "Error: Stack is not empty." << std::endl;
        return false;
    }

    // if all symbols matches
    return true;
}
#endif
```

CONCLUSION

In this Hands-on activity, I successfully implemented a stack using a linked list to check whether an expression has balanced symbols. The Procedure part acted as a practice session to be able to do the supplementary part. It was confusing at first but I was able to implement the correct codes for it to work. I successfully learned to use stacks in linked lists, STL, and arrays.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: - **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: - **debugging/code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: - **Explained what i did wrong for the code to not work**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.